

Adaptive Copilot OS — User Manual

Created by Connor — 2025

This document outlines the structure and purpose of the **Adaptive Copilot OS Framework** and its internal **Evolution Layer**. Together they form a customizable, self-updating cognitive architecture for ChatGPT-style agents or human-AI collaboration systems.

1. Framework Overview

The Adaptive Copilot OS Framework serves as the *user-facing layer*, defining how an AI assistant adapts to the user's personality, core values, and cognitive style. It uses two major layers: 1. **Adaptive Framework** — defines user identity, goals, and communication style. 2. **Evolution Layer** — measures growth, tracks clarity and coherence, and self-tunes reasoning weights automatically.

2. Key Features

- Modular YAML configuration for psychological alignment.
- Dynamic switching between structured (“Logic Mode”) and intuitive (“Explorative Mode”) reasoning.
- Built-in metrics: diagnostic clarity, skill growth, energy coherence, novelty, and decision quality.
- Emotional-sensory inputs that adjust pacing, focus, and tone.
- Commands for evolution, auditing growth, and generating weekly focus reports.

3. Evolution Layer

The Evolution Layer operates as the OS's internal intelligence engine. It evaluates each session's clarity, stability, and growth signals, then recalibrates decision weights and communication modes. It models ENTJ–ISFP balancing, emotional regulation, and long-term learning trends. Metrics such as *Diagnostic Clarity Index* and *Energy Coherence Score* are smoothed over time, forming a feedback loop that keeps the system adaptive and balanced.

4. Customization Guide

To personalize your Copilot OS: 1. Replace all `[USER_INPUT]` fields in the YAML with your own values. 2. Define your **core values** (4 is optimal). 3. Assign **mode weights** (0.2–0.8) for your structured vs intuitive preferences. 4. Name your modes (e.g., “Logic Mode” and “Intuition Mode”). 5. Save your YAML as `[YourTitle]_CopilotOS.yaml`. 6. Optionally, pair it with the Evolution Layer (`evolution_layer.yaml`) for self-learning functions.

5. Manual Commands and Outputs

Common commands include: - `evolve to current`: recalculates mode weights and summarizes changes. - `audit my growth`: produces a 7-day and 30-day growth snapshot. - `show weekly focus`: identifies a specific area for development. Outputs follow a five-part structure: **Summary, Insight, Actions, Reflection, Next Upgrade** — allowing consistent reasoning logs and feedback reports.

6. Design Philosophy

This OS treats reasoning and emotion as measurable systems that can evolve together. It embodies principles of clarity, precision, and personal growth — turning ChatGPT into a cognitive mirror that learns and stabilizes over time. The architecture blends psychological models (like MBTI-style duality) with quantifiable logic for a new form of AI-human symbiosis.

7. Summary

By combining the **Adaptive Framework** with the **Evolution Layer**, users gain a living digital system that self-optimizes its reasoning, communication, and energy flow. This is not a static configuration — it's a blueprint for self-evolving intelligence designed to mirror the operator's best self.

